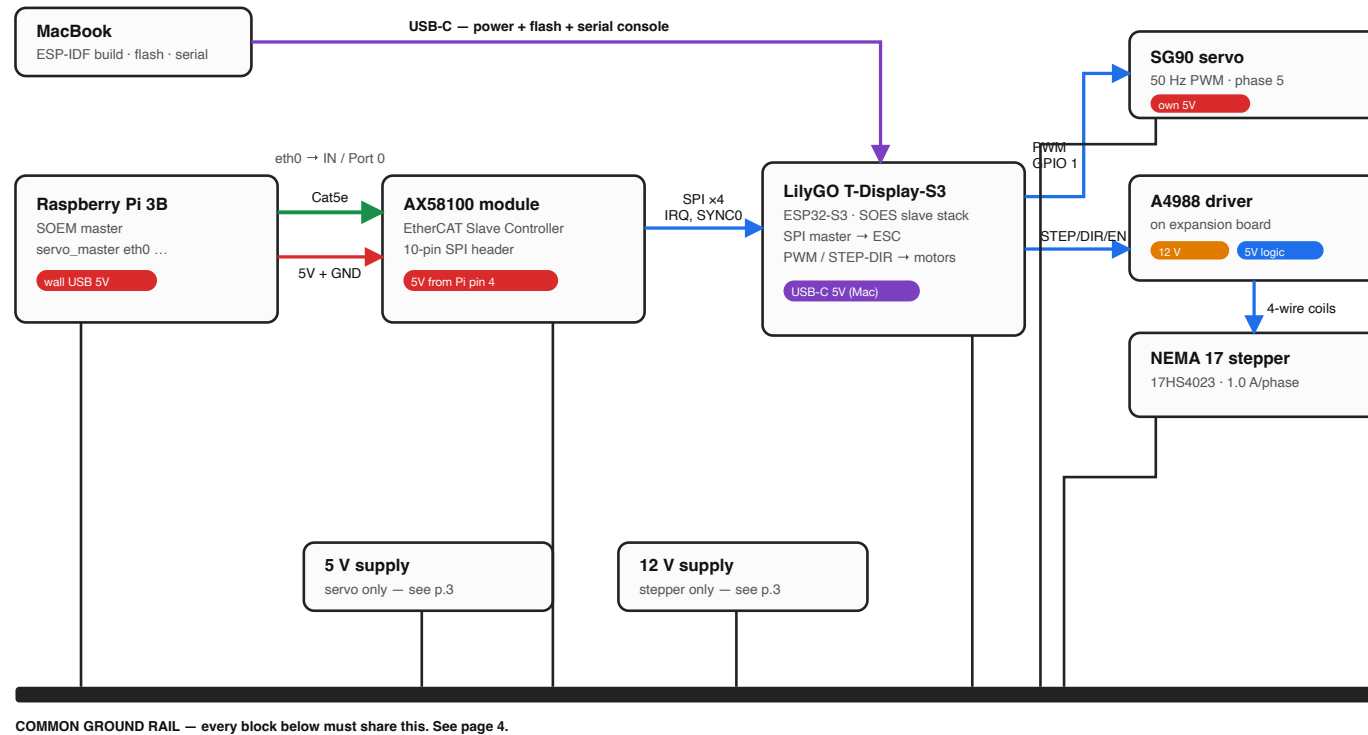


EtherCAT Servo Node — Complete Wiring

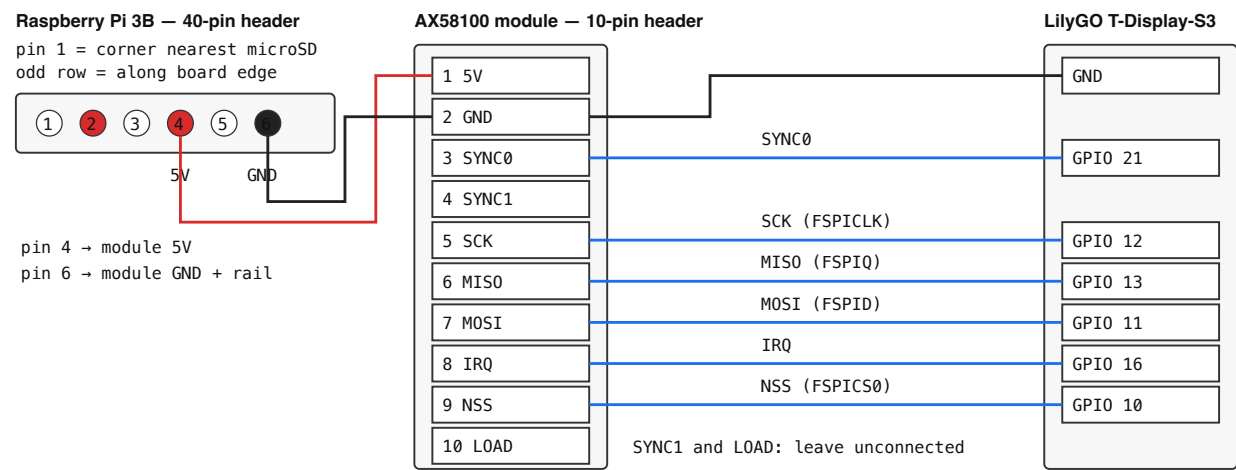
Raspberry Pi 3B (SOEM master) → AX58100 ESC → LilyGO T-Display-S3 (ESP32-S3) → SG90 / NEMA 17 · Page 1 of 4: System overview



Three independent power sources feed this system: the Pi's wall adapter (which also powers the ESC), the Mac's USB-C (which powers the ESP32), and the motor supply. They are **only** tied together by the common ground rail. That single shared ground is what makes every signal in this diagram meaningful — see page 4.

Page 2 — Pi → AX58100 → ESP32-S3

The EtherCAT and SPI section. Build and verify this before any motor is connected.



Connection table

Module pin	Goes to	Wire
1 · 5V	Pi header pin 4 (5V)	red
2 · GND	Pi pin 6 + ground rail	black
3 · SYNC0	ESP32 GPIO 21	blue
4 · SYNC1	— not connected	—
5 · SCK	ESP32 GPIO 12	blue
6 · MISO	ESP32 GPIO 13	blue
7 · MOSI	ESP32 GPIO 11	blue
8 · IRQ	ESP32 GPIO 16	blue
9 · NSS	ESP32 GPIO 10	blue
10 · LOAD	— not connected	—
Pi eth0	Module IN / Port 0 jack, Cat5e	—

Verify before proceeding: firmware reads ESC register `0x0000` over SPI and gets `0xc8` — the same value the Pi already reads over EtherCAT.

Why these pins

GPIO 10–13 are the ESP32-S3's native **FSPI (SPI2) IOMUX** pins, so SPI runs at full speed with no GPIO-matrix routing penalty.

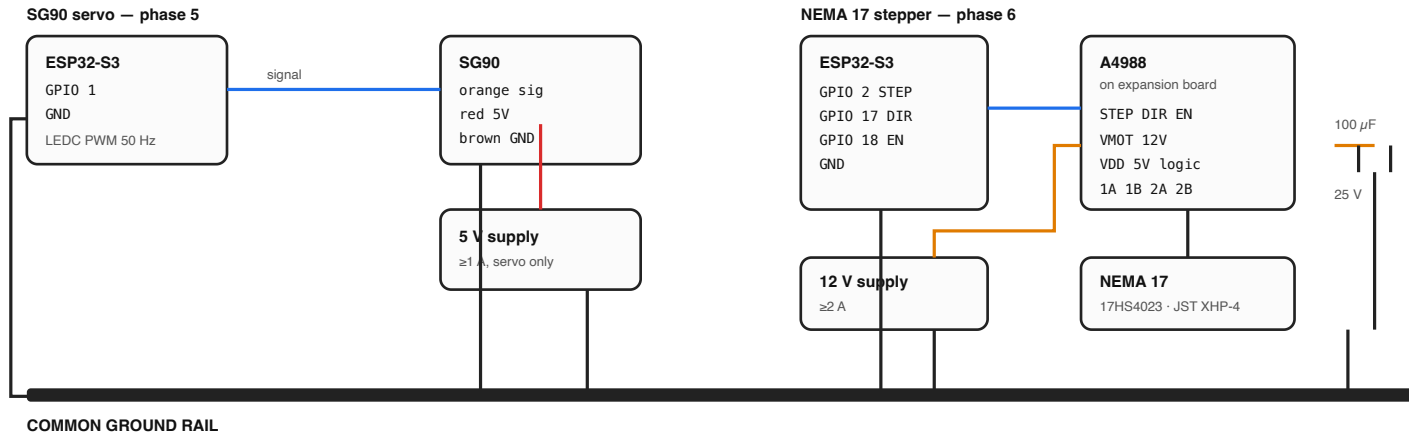
Setting	Value	Source
SPI mode	3 (CPOL=1, CPHA=1)	reg 0x0150
Bit order	MSB first	datasheet
Chip select	active LOW	reg 0x0150

Already confirmed on your hardware (read live over EtherCAT with `servo_master eth0 regs`): PDI Control = `0x05` (SPI Slave), so the `SCS_FUNC` pull-up is already fitted on-board. **You do not need a 4.7K resistor.**

MISO / MOSI are adjacent on the header. Swapping them is the classic mistake — it fails silently, reading `0x00` forever with no other symptom. Drive **NSS as a plain GPIO**, not hardware CS: the ESC needs it held low across a whole multi-byte transaction.

Page 3 — Motors

SG90 first (phase 5), NEMA 17 later (phase 6). Both can stay wired at once — they use different GPIO.



ESP32-S3 pin budget

GPIO	Use	Phase
10, 11, 12, 13	SPI to ESC (NSS/MOSI/SCK/MISO)	4
16	ESC IRQ	4
21	ESC SYNC0	4
1	SG90 PWM signal	5
2	Stepper STEP	6
17	Stepper DIR	6
18	Stepper EN (active low)	6
3, 43, 44	avoid — strapping / UART0	—

3.3 V logic into a 5 V board is fine here. The A4988's logic-high threshold is 2.0 V, so the ESP32's 3.3 V STEP/DIR/EN signals register correctly. Feed the board's VDD logic rail 5 V; no level shifter needed.

Setting the A4988 current limit — do this before the motor spins

Turn the trimpot and measure V_{ref} between the pot's metal top and GND with the multimeter, 12 V applied, motor **disconnected**:

Sense resistor (marked on board)	Formula	Target for 1.0 A
$R100 = 0.1 \Omega$	$V_{ref} = I \times 0.8$	0.80 V
$R050 = 0.05 \Omega$	$V_{ref} = I \times 0.4$	0.40 V

Fit the 100 µF capacitor across VMOT and GND before first power-up. Inductive spikes from the coils will destroy an A4988 otherwise — this is the single most common way these boards die, and both the Allegro datasheet and Pololu's notes call it out explicitly.

Never unplug the motor while 12 V is on. Same failure mode.

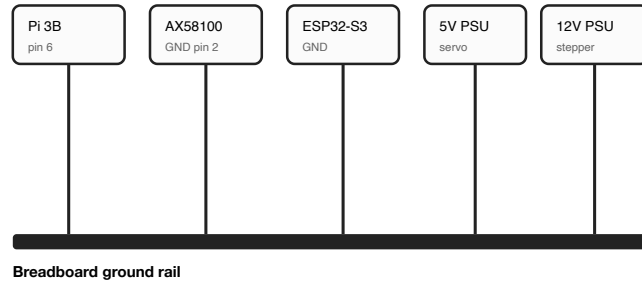
Identifying the coil pairs

Set the multimeter to ohms. Two wires reading about **4.1 Ω** are one coil; across different coils reads open. Your 17HS4023's datasheet phase resistance is 4.1 Ω, so this is a definitive check — do not trust wire colour. Pair them into 1A/1B and 2A/2B. If the motor buzzes without turning, swap one pair.

Page 4 — Ground, power sequence, pre-flight

The page that prevents most of the debugging.

Ground topology



This is the wire people forget. The ESP32 is powered from your Mac's USB-C while the ESC runs off the Pi. Two supplies with no shared reference means the SPI signals have no valid return path, and you read garbage. Symptom: **SPI returns all 0x00 or all 0xFF**, while the wiring looks perfectly correct.

Never feed 5 V into the module from two places. It already takes 5 V from Pi pin 4. Do not also connect the ESP32's 5 V pin to it.

Bring-up order — one new variable at a time

Phase	Milestone	Pass condition
4a	SPI link to ESC	Reads register <code>0x0000 = 0xc8</code>
4b	SOES integrated	<code>set 90</code> → <code>echo=90</code> , slave in OP
5	SG90 moves	<code>set 0/90/180</code> , then <code>sweep</code>
6	NEMA 17 moves	Same commands drive the stepper

Do not wire the motors until 4a passes. Adding a driver, a 12 V rail, current tuning and new firmware at once turns one unknown into five.

Power-up order

1. Everything off. Pi shut down *and unplugged* — GPIO pin 4 stays live after poweroff while the adapter is in.
2. Connect **all grounds first**.
3. Connect signal wires.
4. Power the **Pi** → module LEDs come on, `eth0` links.
5. Power the **motor supply** (12 V / 5 V).
6. Plug the **ESP32 USB-C last**.

Power down in reverse — USB-C out first.

The ESP32 goes last because it **drives** signals into the ESC and the A4988. Driving a pin high on an unpowered chip pushes current through its protection diodes — that is how boards get damaged before any software is even written.

Pre-flight checks

Check	Expected
Continuity: every GND point to rail	beeps / $\sim 0\ \Omega$
Module 5V pin to GND	$\approx 5\text{ V}$, never shorted
MISO and MOSI not swapped	trace both by hand
100 μF cap polarity	stripe = negative → GND
12 V never touches ESP32 or ESC	A4988 VMOT only
Coil pairs by resistance	$\approx 4.1\ \Omega$ within a pair
Vref set with motor disconnected	0.80 V (R100)